

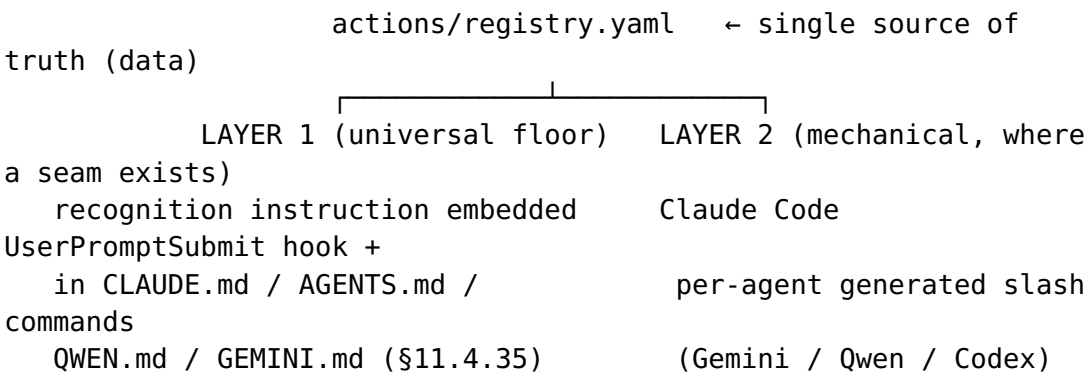
Action-Prefix System — Developer / Maintainer Manual

Field	Value
Revision	1
Created	2026-06-09
Last modified	2026-06-09T00:00:00Z
Status	active
Scope	developer/maintainer manual for the §11.4.140 action-prefix system
Audience	constitution maintainers + consuming-project engineers
Inputs	docs/research/action_prefix_system/ {DESIGN.md, GRAMMAR_ADDENDUM.md, IMPLEMENTATION_REPORT.md, RULE_DRAFT.md}

Anti-bluff (§11.4.6): every component, path, and behaviour below was confirmed against the files present in the constitution submodule at authoring time. Anything designed-but-not-yet-implemented is marked **(spec-only)** with a pointer to what remains. No behaviour is described as working unless the code that performs it exists.

1. The two-layer architecture

The system is **two layers over one shared registry**. The registry (actions/registry.yaml) is the single source of truth; both layers read it.



LAYER 1 — universal, always-on, self-applied

A short “Action-Prefix Recognition” instruction block is mirrored, **verbatim and identical**, into every agent context carrier the constitution maintains: CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md. Each CLI agent loads its own carrier with every prompt (per §11.4.35), reads the instruction, and applies the prefix expansion **itself**. Because the text is identical across all four carriers, the system works on every agent with zero extra setup — this textual identity IS the LAYER-1 universality.

The canonical block lives at actions/recognition_instruction.md, between the markers:

```
<!-- action-prefix-recognition:begin -->
... the verbatim block ...
<!-- action-prefix-recognition:end -->
```

The block is **self-contained**: it inlines the BACKGROUND expansion so the most-used action works even on an agent with no registry-file access, while the registry remains the extensible source of truth for every other action.

Embedding the block into the *live* CLAUDE.md / AGENTS.md / QWEN.md / GEMINI.md is the conductor’s §11.4.26 job — those governance files are conductor-owned (§11.4.58 contention paths). The recognition-instruction file holds the canonical text to embed.

LAYER 2 — mechanical, where the agent exposes a pre-submit seam

- **Claude Code** exposes a UserPromptSubmit pre-submit seam. The hook scripts/hooks/action_prefix_expand.sh reads the prompt, runs the expander, and on a registered match injects the expansion via additionalContext (the spec-guaranteed additive contract — see §3). This is the §11.4.109 anti-forgetting pattern applied to prompt prefixes: the expansion holds even if the model’s recall lapses.
 - **Gemini / Qwen / Codex** expose generated/custom commands but no transparent pre-submit rewrite. The generator scripts/generate_agent_prefix_commands.sh emits a per-agent slash command (/background) from each registry action — the mechanical convenience.
 - **Cursor / Aider / Cline / Continue / Roo / Copilot** have no pre-submit rewrite seam; they are covered by LAYER 1 only (honest §11.4.3 SKIP of LAYER 2 — LAYER 1 fully covers the free-form form).
-

2. Component inventory (verified present)

All paths are relative to the constitution submodule root.

Path	Role	Layer
actions/registry.yaml	single source of truth: action → expansion + grammar + rules	shared
actions/ recognition_instruction.md	canonical LAYER-1 recognition block (marked for embedding into the 4 carriers)	1
scripts/action_prefix_lib.sh	shared pure expander library (apx_* API)	2 + tests
scripts/hooks/ action_prefix_expand.sh	Claude Code UserPromptSubmit hook (mechanical)	2
scripts/ generate_agent_prefix_commands.sh	per-agent slash-command generator (Gemini/Qwen TOML, Codex md)	2
scripts/install_action_prefix.sh	idempotent installer — wires the hook by reference + runs the generator	install seam
actions/generated/ {gemini,qwen,codex}/background. {toml,md} + README.md	generated slash-command artefacts (generator output)	2 (generated)

All four scripts are `chmod +x`, and pass both `sh -n` and `bash -n` (§11.4.67). The shebangs honestly declare `#!/usr/bin/env bash` because the scripts use bash arrays / `[[ ]]` / `BASH_SOURCE` — `bash -n` is the authoritative parse gate.

Decoupling (§11.4.28). The registry + scripts carry NO project-specific data. A consuming project ships its own actions/registry.yaml OR inherits the constitution default by reference (the §11.4.80 `codegraph_*` pattern). Every script reads the registry path from `$HELIX_ACTION_REGISTRY` (default: `constitution/actions/registry.yaml`) — never a hardcoded project path. This is proven by an alt-registry test (Implementation Report §4).

3. How LAYER 1 + LAYER 2 work (mechanics)

LAYER 1 (recognition instruction)

On every prompt the agent loads its carrier (which contains the recognition block). When the prompt's first non-blank line matches the grammar, the agent itself: (1) consults `actions/registry.yaml` (or has the `BACKGROUND` expansion inlined in the block); (2) replaces the `ACTION_NAME :: prefix` with the expansion text and applies the rules; (3) executes the remainder. No process runs — this is the model obeying its loaded instruction.

LAYER 2 (Claude Code hook)

`scripts/hooks/action_prefix_expand.sh` is wired under `hooks.UserPromptSubmit` in `.claude/settings.json`. On each prompt:

1. it reads the event JSON on **stdin** and extracts `.prompt` (with a `no-jq` fallback, mirroring the existing `guard-forbidden-commands.sh`);
2. it runs the shared expander (`scripts/action_prefix_lib.sh`, `apx_expand_prompt`) against the prompt + registry;
3. on a **registered match** it prints, on stdout:

```
{"hookSpecificOutput":  
{"hookEventName":"UserPromptSubmit","additionalContext":"<expansion + residual>"}}
```

and exits 0 — Claude receives the expansion as injected context and obeys it (the spec-guaranteed **additive** behaviour: `UserPromptSubmit` stdout / `additionalContext` is *added* to the model's context, per Claude Code's hooks documentation — see `RESEARCH.md` §1.1);
4. on **no match / escape** it exits 0 with empty output (pass-through);
5. on an **unknown grammar-shaped token** it injects an `additionalContext` clarify note (asking which registered action

was meant per §11.4.66 / §11.4.105) — it NEVER invents an expansion (§11.4.6);

6. on any **internal error** it FAILS OPEN to pass-through — a prefix system must never reject a user's prompt.

The pure expander scripts/action_prefix_lib.sh is reused by **three** consumers — the Claude hook, the tests, and the slash-command generator — so there is one tested implementation. Its apx_expand_prompt emits a JSON verdict (expand / noop / escape / ask) implementing the full grammar: first-non-blank-line anchoring, UPPERCASE-only token, exact :: separator, outer-to-inner stacked-prefix recursion, leading-escape, and unknown-token→ASK.

YAML reading (+ honest fallback)

- **Primary:** python3 + PyYAML for robust validate / list / expansion.
- **Fallback:** an awk/sed reader used automatically when python3/PyYAML is absent. It is **honestly narrow** — it targets exactly the registry shape this project emits (schema_version:, grammar.prefix_regex:, actions[] with - name: + a folded expansion: >- block scalar or an inline expansion:). It is NOT a general YAML parser. It was verified byte-identical to the python path for validate/list/expansion/expand/escape/unknown/lowercase, and for first/last/inline/missing actions in a multi-action registry.

The JSON emitted by the library + hook is produced/escaped by hand (apx__json_escape / apx__emit_json), so no JSON-writer dependency is needed; jq is preferred for reading our own emitted JSON, with a sed fallback.

4. The registry schema

actions/registry.yaml — the single source of truth and the extension contract.

4.1 Implemented schema (current registry)

```
schema_version: 1
grammar:
  prefix_regex: '^[A-Z][A-Z0-9_]*' :: '      # anchored, uppercase
              token, " :: " separator
  case_sensitive: true                      # action names are
              UPPERCASE-only
  multiple_prefixes: stack                  # "A :: B :: rest"
              applies A then B
  escape: '\                                # "\BACKGROUND :: x"
              → literal, no expansion
```

```

actions:
- name: BACKGROUND
  version: 1
  summary: >-
    Run the remainder of the prompt as a background, subagent-
    driven work
    stream in parallel with all main work, producing rock-solid
    physical proof.
  expansion: >-
    The following prompt that we will provide MUST BE executed
    in background in
    parallel with all main work streams using the subagents-
    driven development
    approach! All work done MUST PRODUCE rock solid evidence
    covered with hard
    physical proof(s) that all done is working as expected and
    as specified
    without any false results and without any bluff!
  rules:
    - "The expansion REPLACES the 'BACKGROUND :: ' prefix; the
      remainder of the prompt is the actual task to perform."
    - "Compose with §11.4.70 / §11.4.20 (subagent-driven),
      §11.4.58 / §11.4.103 (parallel streams), §11.4.89
      (background execution), §11.4.5 / §11.4.69 / §11.4.107
      (captured physical evidence), §11.4 (anti-bluff)."
```

composes_with:

```

["11.4.20", "11.4.58", "11.4.70", "11.4.89", "11.4.103", "11.4.5", "11.4.69", "11.4.107"]

```

4.2 Field semantics

Field	Required	Meaning
schema_version	yes	integer; bumps on an incompatible schema change
grammar.prefix_regex	yes	the anchored match; the SAME string both layers honour
grammar.case_sensitive	yes	true — action names are UPPERCASE only
grammar.multiple_prefixes	yes	stack — apply outer-to-inner, left-to-right
grammar.escape	yes	leading \ makes the prefix literal (no expansion)
actions[].name	yes	<code>^[A-Z][A-Z0-9_]*\$</code> — the trigger token

Field	Required	Meaning
actions[].version	yes	per-action monotonic integer (audit + change tracking)
actions[].summary	yes	one-line human description (≥ 6 words per §11.4.91)
actions[].expansion	yes	the verbatim text that REPLACES the prefix
actions[].rules	no	free-text behavioural constraints applied after expansion
actions[].composes_with	no	constitution clauses the action binds (audit)

4.3 Namespace / slash schema additions (spec-only)

The GRAMMAR_ADDENDUM adds the 4-form grammar (namespaced `::` + slash forms). These keys are **designed but NOT yet present** in the registry or honoured by the library:

```
grammar:
  default_namespace: DEFAULT
  namespace_separator: '::'      # inside the token, no spaces
  body_separator: ' :: '        # token <-> rest, spaces both
                                sides ('::' form)
  slash_prefix: '/'
  slash_body_separator: ' '      # token <-> rest (slash form)
actions:
  - name: BACKGROUND
    namespaces:
      [DEFAULT]                # which namespaces this action is
                               registered under
    slash_bare: auto            # bare /BACKGROUND honored
                               unless host-command collision
    slash_conflicts: []         # known colliding host slash
                               commands (forces /DEFAULT::BACKGROUND)
```

When these land, `apx_parse_prefix` must recognise all 4 forms (returning `namespace|DEFAULT, action, rest, matched_form`), `apx_expand_prompt` must resolve `namespace+action` and honour the bare-slash conflict rule, the LAYER-2 hook must handle all 4 first-

line forms, and the generator must additionally emit a namespaced /default::background artefact (or a default-background alias where the agent's command syntax forbids ::).

5. Install / wiring (install_action_prefix.sh, by reference)

scripts/install_action_prefix.sh is the §11.4.75 out-of-the-box install seam. Run as part of the consuming project's setup. It:

1. Merges the UserPromptSubmit hook entry into the consuming project's .claude/settings.json **by reference** — a relative constitution/... path when the constitution is a subdir, never copying the hook (§11.4.80). This is idempotent: a re-run produces no duplicate entry, and pre-existing hooks (e.g. a PreToolUse guard) are preserved.
2. Backs up settings.json before editing.
3. Runs scripts/generate_agent_prefix_commands.sh to (re)produce the per-agent slash commands.

By-reference, never copy is the rule (§11.4.80): the consuming project points at the constitution's hook/scripts, so a constitution update propagates without a per-project re-copy.

6. How to add a NEW action (step-by-step)

The whole extensibility contract is: **new action = new registry row**. No code change in either layer (the registry is data, not code).

1. **Add one actions[] row** to actions/registry.yaml:

```
- name: NEWACTION                                # ^[A-Z][A-Z0-9_]*$, UPPERCASE
  version: 1
  summary: >-                                     # ≥6 words / ≥40 chars per
    §11.4.91 – name SUBJECT + GOAL
    One-line human description of what this action does to the
    prompt.
  expansion: >-                                    # the verbatim text that
    REPLACES the prefix
    The instruction that the remainder of the prompt runs
    under.
  rules:                                           # optional
    - "Free-text behavioural constraint applied after
      expansion."
  composes_with: ["11.4.X", "..."] # optional – constitution
    clauses it binds
```

2. Regenerate the per-agent slash commands:

```
bash scripts/generate_agent_prefix_commands.sh
```

This re-emits actions/generated/{gemini,qwen,codex}/newaction.{toml,md} so /newaction appears on every agent that supports generated commands. (No change to the hook or the library is needed.)

3. (Optional) Inline the expansion in the LAYER-1 block

ONLY if the action must work out-of-the-box even without registry-file access (like BACKGROUND is inlined). Otherwise the registry alone suffices — the recognition instruction tells the agent to consult it.

4. Add coverage per §11.4.4(b) / §1.1: a unit test (registry parses, grammar matches, expansion correct) + the paired meta-test mutation (corrupt the new expansion or strip a key → the gate FAILs). See docs/research/action_prefix_system/TEST_PLAN.md.

5. Sync docs — bump the §11.4.44 revision header on touched docs; the §11.4.65 export pipeline regenerates the .html/.pdf siblings at commit time (the conductor runs the exporters — do not run them mid-build).

Verify the new expansion is byte-faithful: the generated slash command's prompt body must equal the registry expansion verbatim. The generator copies it directly, so a mismatch indicates the registry was edited but the generator not re-run.

7. The conflict rule for the bare slash form (spec-only)

For form 3 (/ACTION_NAME), the bare slash is honoured as the action ONLY when ACTION_NAME (case-folded for the collision check) does **not** match a built-in / host slash command of the agent. The registry carries (when the namespace schema lands) a per-action slash_bare: true|false|auto and a slash_conflicts: [...] list:

- slash_bare: auto — enable the bare /ACTION_NAME unless a known host command collides.
- Form 4 (/PREFIX::ACTION_NAME, e.g. /DEFAULT::BACKGROUND) is **always unambiguous and always honoured** — it is the safe disambiguation when a collision exists.

This rule exists because different agents ship different built-in slash commands; `/background` may collide on some agent's command set. The namespaced form is the collision-proof escape hatch.

Status (§11.4.6): today the generator emits the bare `/background` (form 3) only. The conflict-aware `slash_bare/slash_conflicts` keys and the namespaced `/default::background` artefact are spec-only until the namespace schema lands (see §4.3).

8. Troubleshooting

Symptom	Cause / check	Fix
<code>BACKGROUND :: ...</code> not expanded on Claude Code	Hook not wired	Run <code>bash scripts/install_action_prefix.sh</code> ; confirm a <code>UserPromptSubmit</code> entry pointing at <code>constitution/scripts/hooks/action_prefix_expand.sh</code> exists in <code>.claude/settings.json</code>
Prefix not recognised on a non-Claude agent	LAYER-1 block not in that agent's carrier	Ensure the <code><!-- action-prefix-recognition:begin/end --></code> block is present in the agent's carrier (<code>CLAUDE.md/AGENTS.md/QWEN.md/GEMINI.md</code>); the conductor lands it via §11.4.26
<code>/background</code> missing on Gemini/Qwen/Codex	Generator not run, or commands not installed	Run <code>bash scripts/generate_agent_prefix_commands.sh</code> ; the installer wires the generated files to <code>.gemini/commands/</code> / <code>.qwen/commands/</code> / <code>prompts/</code> by reference
A typo like <code>BACKGROUND :: x</code> was treated literally instead of asking	Working as designed — unknown token → ask/literal, never guess (§11.4.6)	Use the exact registered token; the agent names the closest match per §11.4.66 / §11.4.105
<code>BACKGROUND::do X</code> (no spaces) not expanded	Wrong separator — form 1 needs exactly <code>::</code>	Use <code>BACKGROUND :: do X</code> (space-colon-colon-space)
please <code>BACKGROUND :: x</code> not expanded	Prefix not at first-non-blank-line start	Put the keyword at the very start of the first non-blank line
Custom registry not used	<code>\$HELIX_ACTION_REGISTRY</code> not set	Export <code>HELIX_ACTION_REGISTRY=/path/to/your/registry.yaml</code>
Hook seems to reject prompts	It should never — it FAILS OPEN to pass-through on error	Inspect the hook's <code>stderr</code> ; an internal error is a bug to file, not expected behaviour

Symptom	Cause / check	Fix
DEFAULT::BACKGROUND :: ... (form 2) or / DEFAULT::BACKGROUND (form 4) not working	Forms 2/4 are spec- only (§4.3)	Use form 1 or form 3 today; namespace forms are a tracked follow-up

To validate the registry directly:

```
# (uses python3+PyYAML if present, else the narrow awk fallback)
bash -c 'source scripts/action_prefix_lib.sh &&
        apx_validate_registry && echo OK'
```

9. Related documents

- [USER_GUIDE.md](#) — end-user guide (4 forms, BACKGROUND effect, escape, per-agent matrix, quick-start).
- [DIAGRAMS.md](#) — Mermaid diagrams (expansion flow, two-layer architecture, 4-form decision tree, add-a-new-action sequence).
- docs/research/action_prefix_system/{RESEARCH.md, DESIGN.md, GRAMMAR_ADDENDUM.md, IMPLEMENTATION_REPORT.md, RULE_DRAFT.md, TEST_PLAN.md} — source design + test plan.
- Canonical authority: Constitution.md §11.4.140.